

StudyOS Thesis-Finder

Minimal-maintenance discovery today; live candidate discovery tomorrow

StudyOS Team

July 2026

Department of Computer Science, University of Tübingen

The problem, from both sides

Students

- Spend weeks figuring out which chairs even work on their interests
- See 4–5 listed topics, assume that is the whole lab, and never reach out
- Send generic cold emails to the wrong chairs

Professors

- Receive far more inquiries than they can supervise
(one chair: 50+ inquiries for 20 slots)
- Lose time on rejection emails for mismatched inquiries
- Get low-signal first contacts that miss the chair's actual research

Students and professors both lose when the first contact is poorly matched.

What 27 professors told us — before we built anything

We interviewed 27 professors in the CS department. The verdict on a classic *topic-listing platform* was blunt or reluctant:

- ~**50%** do not want a central platform — either already overrun with applicants, or convinced topics must be *co-created in conversation*, not browsed from a list
- ~**35%** would try a tool — but only if zero-friction, with no reminder emails, and stable for years
- ~**15%** recruit through other channels entirely

“Profs never update the topics. We’ve had many attempts in the department already.”

A topic platform was tried here around 2022 and failed — for **incentive** reasons, not UI reasons.

The plan we started with — and why we abandoned it

First attempt

A hosted web app — FastAPI · Celery · Postgres+pgvector · React — scraping professors and serving a match portal.

- Covered **CS Tübingen only**, would have gone stale within months
- Assumed HiWis would keep a scraping pipeline alive “forever”, but with low effort
- Still needed someone to maintain brittle scraped state

The pivot

A maintained database is the wrong asset.

The professors' own signal pointed the way: *build for students, read what already exists, ask chairs for nothing.*

So instead of maintaining scraped state, we packaged *the search process*: today with a small reviewable backbone, and next with runtime candidate discovery.

The idea: ship the method, not the data

Zero professor participation. Minimal maintained state. Live verification at runtime.

Setup

Copy-and-paste the skill folder into a standard AI agent. No account, custom app, or professor-facing admin workflow.

The current backbone is a small bridge: inspectable anchors plus live checks, not a large database of stale rows.

Student path

1. **Formulate:** build a six-dimension profile
2. **Discover:** find chairs, groups, and companies with evidence
3. **Prepare:** draft a specific first contact

Deliverable: a **map of the possibilities** — what exists, why it fits, and where to start.

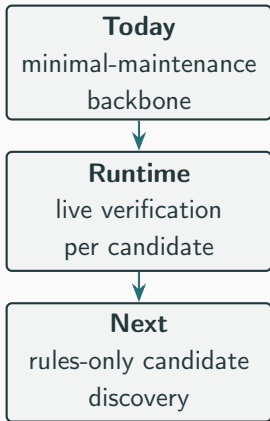
How it works: one flow, specialized skills



Discovery delegates to [find-university-chairs](#) and/or [find-company-thesis-options](#). If needed, [find-recent-papers](#) enriches the final proposal and contact draft.

design-agent-skill is the meta-skill for redesigning or adding skills; it is not part of the student-facing path.

The core IP: a search method, not a store



The current system is not a classic database, but it is not completely data-free either:

- **Small backbone** — a reviewable set of structural anchors for faculties and BW companies
- **Search strategy** — profile-to-query mappings, quality filters, dedup, and routing rules
- **Live verification** — URLs, people, thesis signals, and evidence are checked during the run

The next version should replace stored anchors with **rules-only candidate discovery** across multiple source axes.

Why not a maintained database

Classic database	Current backbone	Future discovery
High maintenance	Minimal maintenance	Rules, not stored entities
Full of stale rows	Small reviewable anchors	Fresh candidates per profile
Needs backend/admin	Plain Markdown	Live search + verification
Hard to port	Portable skill package	Portable when search tools exist

The honest claim is not “no data”. It is: keep durable knowledge in the method, and verify volatile facts live.

- **Student data never leaves the session.** The profile (interests, coursework, constraints) exists only in the student's own agent conversation — nothing is stored or transmitted by us.
- **Only public academic data is used:** names, roles, research areas, and publications as published by chairs themselves — GDPR-conscious by design.
- **Everything is inspectable.** Skills and references are plain text in a public repository; therefore easily adaptable by hand or AI agent.
- **No fabricated numbers.** A hard project rule: no placeholder metrics, no invented team sizes or citation counts.

Where we are, and what comes next

Behind us

- Interviewed 27 professors
- Dropped the full-stack app
- Shipped the skill-based tool
- Got some feedback by students.

Right now

- Minimal-maintenance backbone as bridge
- Small, inspectable anchors
- Live verification during runs

Next

- Dedicated candidate-discovery skills
- Rules for where and how to search
- Compare via command simulations
- upload to the big providers skill repositories

Backbone today means pragmatic anchors; backbone tomorrow means discovery rules or alternatively some kind of automatic maintenance.

1. **Nothing, by default.** The tool works from public pages and publications as they are.
2. **Optionally:** a few research-area keywords, or a plain “looking for / not looking for” note.
3. **(Almost) no maintenance.** No scraping, no database, no admin workflow, if our idea of self-maintaining discovery is working.

Students get better discovery. Professors get fewer but better inquiries.
Nobody is forced into a professor-facing admin workflow.

Thank you — questions?